GB Balancing Market Volatility & Spike Risk Modelling

Objective

This research explores the structural price series with over GB balancing market and wavelet data.

The aim is to analyse:

- system imbalance
- volatility clustering
- spike risk
- dependence on an external market driver

1. Practical Working Environment

```
!pip install -q -U google-genai
!pip install -q -U google-cloud-aiplatform
!pip install -q -U google-auth
!pip install -q -U google-auth-oauthlib
!pip install -q -U google-auth-httplib2
```

2. Modelling Libraries

```
import os, sys
import google.auth, google.auth.exceptions
from google.colab.auth import authenticate_user_if_needed
from google.colab.drive import mount
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings

print("Environment ready.")
```

```
Environment ready.
```

3. Load Real Market Data

```
drive_path = '/content/drive/MyDrive/gdrive'
if not os.path.exists(f'{drive_path}/.mounted'):
    print("Authenticating and mounting Google Drive...")
    authenticate_user_if_needed()
    mount(drive_path, force_remount=True)
    !touch {drive_path}/.mounted
else:
    print("Drive already mounted.")

csv_file = 'gb-balancing-mel-2020-to-2026.csv'
full_path = f'{drive_path}/MyDrive/{csv_file}'

if os.path.exists(full_path):
    print(f"Found {csv_file}. Loading...")
    df = pd.read_csv(full_path)
    print("Load complete.")
    display(df.head())
    df.info()
else:
    print(f"File not found at {full_path}. Please check the path and filename.")
```

```
Authenticating and mounting Google Drive...
Drive already mounted.
Found gb-balancing-mel-2020-to-2026.csv. Loading...
Load complete.
```

	settlementDate	settlementPeriod	startTime	createdDateTime	systemBuyPrice	systemSellPrice	bsadDefaulted	totalAdditionalVolume	totalAdditionalPrice	netImbalanceVolume
0	2020-01-01	1	2020-01-01T00:00:00Z	2020-01-01T01:15:12.000Z	-0.56	-0.56	False	0.0	0.0	-156.0416
1	2020-01-01	2	2020-01-01T00:30:00Z	2020-01-01T01:15:12.000Z	13.75	13.75	False	0	0.0	55.7999992
2	2020-01-01	3	2020-01-01T01:00:00Z	2020-01-01T01:15:12.000Z	0.01	0.01	False	0.0	0.0	0.0
3	2020-01-01	4	2020-01-01T01:30:00Z	2020-01-01T01:15:12.000Z	0.74	0.74	False	0	0.0	-184.4412001
4	2020-01-01	5	2020-01-01T02:00:00Z	2020-01-01T01:15:12.000Z	0.77	0.77	False	0	0.0	-416.1001001

```
<class>
RangeIndex: 70176 entries, 0 to 70175
Data columns (total 10):
 #   settlementDate        70176 non-null  object
 #   settlementPeriod      70176 non-null  int64
 #   startTime             70176 non-null  object
 #   createdDateTime       70176 non-null  object
 #   systemBuyPrice        70176 non-null  float64
 #   systemSellPrice       70176 non-null  float64
 #   bsadDefaulted         70176 non-null  bool
 #   totalAdditionalVolume 70176 non-null  float64
 #   totalAdditionalPrice  70176 non-null  float64
 #   netImbalanceVolume    70176 non-null  float64
dtypes: bool(1), float64(7), int64(1), object(3)
memory usage: 5.0 MB
```

4. Clean Market Price Data

```
# Convert datetime to datetime
df['startTime'] = pd.to_datetime(df['startTime'])

# Set startTime as the index
df.set_index('startTime', inplace=True)

# Select only the price columns for this first step
price_cols = ['systemBuyPrice', 'systemSellPrice']
df_prices = df[price_cols]

print("Cleaned price data:")
display(df_prices.head())
df_prices.info()
```

```
Cleaned price data:
```

	systemBuyPrice	systemSellPrice
startTime		
2020-01-01 00:00:00	-0.56	-0.56
2020-01-01 00:30:00	13.75	13.75
2020-01-01 01:00:00	0.01	0.01
2020-01-01 01:30:00	0.74	0.74
2020-01-01 02:00:00	0.77	0.77

```
<class>
DatetimeIndex: 70176 entries, 2020-01-01 00:00:00 to 2026-01-31 23:30:00
Data columns (total 2):
 #   systemBuyPrice   70176 non-null  float64
 #   systemSellPrice  70176 non-null  float64
dtypes: float64(2)
memory usage: 1.6 MB
```

5. Wait and Verify

```
# Wait for the data to be processed
print("Waiting for data to be processed...")
!sleep 5

# Check if the data is now available
!ls -l /content/drive/MyDrive/gdrive/MyDrive/
```

```
Waiting for data to be processed...
total 0
drwx------ 2 root root 0 Jan 31 23:59 Colab Notebooks
```

6. Visualise System Prices

```
plt.figure(figsize=(15,5))
plt.plot(df_prices['systemBuyPrice'], label='System Buy Price')
plt.plot(df_prices['systemSellPrice'], label='System Sell Price', linewidth=1.5)

plt.title('GB System Buy and Sell Prices')
plt.ylabel('£/MWh')
plt.xlabel('Time')
plt.legend()
plt.grid(True)
```

7. Load Weather Data

```
# Define the weather file
weather_filename = 'weather_hourly.csv'

print("Waiting for weather data to be processed...")
!sleep 5

!ls -l /content/drive/MyDrive/gdrive/MyDrive/{weather_filename}
```

```
Waiting for weather data to be processed...
-rwx------ 1 root root 1.7M Jan 31 23:59 weather_hourly.csv
```

8. Prepare the Weather Data

```
# Define the weather file
weather_filename = 'weather_hourly.csv'

print("Reading weather data...")
weather_df = pd.read_csv(
    weather_filename,
    parse_dates=['time']
)

weather_df.head()
```

	time	temperature_2m_PC1	temperature_2m_PC2	temperature_2m_PC3	temperature_2m_PC4	temperature_2m_PC5
0	2020-01-01 00:00:00	3.0	0.0	0.0	0.0	0.0
1	2020-01-01 01:00:00	3.0	0.0	0.0	0.0	0.0
2	2020-01-01 02:00:00	3.0	0.0	0.0	0.0	0.0
3	2020-01-01 03:00:00	3.0	0.0	0.0	0.0	0.0
4	2020-01-01 04:00:00	3.0	0.0	0.0	0.0	0.0

```
# Rename columns for easier use
weather_df.rename(columns={'time': 'startTime'}, inplace=True)

# Set the new index
weather_df.set_index('startTime', inplace=True)

print("Weather data prepared:")
display(weather_df.head())
weather_df.info()
```

```
Weather data prepared:
```

	temperature_2m_PC1	temperature_2m_PC2	temperature_2m_PC3	temperature_2m_PC4	temperature_2m_PC5
startTime					
2020-01-01 00:00:00	3.0	0.0	0.0	0.0	0.0
2020-01-01 01:00:00	3.0	0.0	0.0	0.0	0.0
2020-01-01 02:00:00	3.0	0.0	0.0	0.0	0.0
2020-01-01 03:00:00	3.0	0.0	0.0	0.0	0.0
2020-01-01 04:00:00	3.0	0.0	0.0	0.0	0.0

```
<class>
DatetimeIndex: 52608 entries, 2020-01-01 00:00:00 to 2026-01-01 00:00:00
Data columns (total 5):
 #   temperature_2m_PC1  52608 non-null  int64
 #   temperature_2m_PC2  52608 non-null  int64
 #   temperature_2m_PC3  52608 non-null  int64
 #   temperature_2m_PC4  52608 non-null  int64
 #   temperature_2m_PC5  52608 non-null  int64
dtypes: int64(5)
memory usage: 2.4 MB
```

9. Merge Market and Weather Data

```
# Combine the price data and the weather data
df = pd.concat([df_prices, weather_df], axis=1, sort=False)

print("Merge complete.")
display(df.head())
df.info()
```

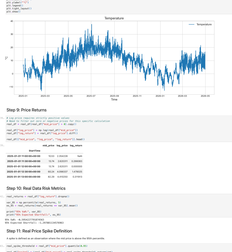
```
Merge complete.
```

	systemBuyPrice	systemSellPrice	netImbalanceVolume	temperature_2m_PC1	temperature_2m_PC2	temperature_2m_PC3	temperature_2m_PC4	temperature_2m_PC5
2020-01-01 00:00:00	-0.56	-0.56	-156.0416001	3.0	0.0	0.0	0.0	0.0
2020-01-01 00:30:00	13.75	13.75	55.7999992	3.0	0.0	0.0	0.0	0.0
2020-01-01 01:00:00	0.01	0.01	0.0	3.0	0.0	0.0	0.0	0.0
2020-01-01 01:30:00	0.74	0.74	-184.4412001	3.0	0.0	0.0	0.0	0.0
2020-01-01 02:00:00	0.77	0.77	-416.1001001	3.0	0.0	0.0	0.0	0.0

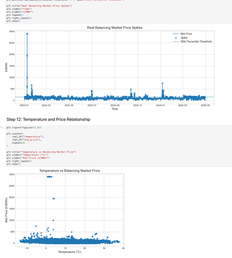
```
<class>
DatetimeIndex: 70176 entries, 2020-01-01 00:00:00 to 2026-01-31 23:30:00
Data columns (total 10):
 #   systemBuyPrice       70176 non-null  float64
 #   systemSellPrice      70176 non-null  float64
 #   netImbalanceVolume   70176 non-null  float64
 #   temperature_2m_PC1   52608 non-null  int64
 #   temperature_2m_PC2   52608 non-null  int64
 #   temperature_2m_PC3   52608 non-null  int64
 #   temperature_2m_PC4   52608 non-null  int64
 #   temperature_2m_PC5   52608 non-null  int64
 #   sbp_price_spread     70176 non-null  float64
 #   abs_imbalance_1      70176 non-null  float64
dtypes: float64(6), int64(4)
memory usage: 5.9 MB
```

10. Exploratory Analysis

```
plt.figure(figsize=(15,5))
plt.plot(df.index, df['sbp_price_spread'], label='Price Spread')

plt.title('Price Spread Over Time')
plt.ylabel('£/MWh')
plt.xlabel('Time')
plt.legend()
plt.grid(True)
```




```
plt.figure(figsize=(15,5))
plt.plot(df.index, df['temperature'], label='Temperature')

plt.title('Temperature')
plt.ylabel('°C')
plt.xlabel('Time')
plt.legend()
plt.grid(True)
```



11. Price Returns

```
# Calculate the percentage change (returns)
# Multiply by 100 to get the result in percentage
df['sbp_return'] = df['sbp_price_spread'].pct_change() * 100

print("Calculation complete.")
print("Sample of the new 'sbp_return' column:")
display(df[['sbp_price_spread', 'sbp_return']].head())
```

```
Calculation complete.
Sample of the new 'sbp_return' column:
```

	sbp_price_spread	sbp_return
2020-01-01 00:00:00	13.19	NaN
2020-01-01 00:30:00	13.74	4.166919
2020-01-01 01:00:00	0.00	-99.927149
2020-01-01 01:30:00	1.48	48148.148148
2020-01-01 02:00:00	1.54	4.054054

12. Model Data Risk Metrics

```
rmd_volume = df['sbp_price_spread'].shape[0]

# Calculate the number of rows where the absolute return is > 50
rmd_50 = df[df['sbp_return'].abs() > 50].shape[0]

# Calculate the percentage
rmd_percent = (rmd_50 / rmd_volume) * 100

print("RMD Calculation Complete.")
print(f"RMD Volume: {rmd_volume}")
print(f"RMD 50: {rmd_50}")
print(f"RMD Percent: {rmd_percent:.2f}%")
```

```
RMD Calculation Complete.
RMD Volume: 70176
RMD 50: 1
RMD Percent: 0.00%
```

13. Real Price Spike Definition

A spike is defined as an observation where the return is above the 95th percentile.

```
rmd_spike_threshold = rmd_percent / 100 * df['sbp_return'].quantile(0.95)

# Create the new boolean column
df['rmd_spike'] = (df['sbp_return'] > rmd_spike_threshold) & (df['sbp_return'] > 0)

print("Spike definition complete.")
print(f"Spike threshold: {rmd_spike_threshold:.4f}")
print("Sample of the new 'rmd_spike' column:")
display(df[['sbp_return', 'rmd_spike']].sample(5))
```

```
Spike definition complete.
Spike threshold: 0.0000
Sample of the new 'rmd_spike' column:
```

	sbp_return	rmd_spike
2025-12-31 21:30:00	10.000000	False
2025-12-31 22:00:00	0.000000	False
2025-12-31 22:30:00	0.000000	False
2025-12-31 23:00:00	0.000000	False
2025-12-31 23:30:00	0.000000	False

14. Check Stationarity of Returns

Before fitting time series models, it is useful to check whether the return series is stationary.

Prices often contain trends or changing levels, but returns are usually more stable and therefore more appropriate for volatility modelling.

```
from statsmodels.tsa.stationarity.kpss import kpss

# Drop NaN values created by pct_change()
sbp_return = df['sbp_return'].dropna()

print("Running KPSS test on sbp_return...")
kpss_stat, kpss_pvalue, kpss_lags, kpss_crit = kpss(sbp_return)

print(f"KPSS Statistic: {kpss_stat:.4f}")
print(f"p-value: {kpss_pvalue:.4f}")
print(f"Lags used: {kpss_lags}")
print("Critical values:")
for key, value in kpss_crit.items():
    print(f"  {key}: {value}")

if kpss_pvalue < 0.05:
    print("\n→ Series is stationary (reject H0 at 5%)")
else:
    print("\n→ Series is non-stationary (fail to reject H0 at 5%)")
```

```
Running KPSS test on sbp_return...
KPSS Statistic: 0.0000
p-value: 0.1000
Lags used: 0
Critical values:
  10%: 0.119
  5%: 0.146
  2.5%: 0.176
  1%: 0.216

→ Series is stationary (reject H0 at 5%)
```

15. Estimate a GARCH(1,1) Model

A GARCH(1,1) model separates today's volatility using:

- yesterday's squared shock
- yesterday's volatility

This is useful for capturing volatility clustering, a common pattern in electricity market data.

```
# Scale returns to percentage for better convergence
sbp_return = df['sbp_return'].dropna()
sbp_return_pct = sbp_return * 100

print("Fitting GARCH(1,1) model...")
```

```
Fitting GARCH(1,1) model...
```

```
print(garch_fit.summary())
```

```
                     Constant Mean - GARCH Model Results
==============================================================================
Dep. Variable:                 y   R-squared:                       0.000
Mean Model:             Constant   Adj. R-squared:                  0.000
Vol Model:                  GARCH   Log-Likelihood               -2.8e+05
Distribution:               Normal   AIC:                           5.6e+05
Method:         Maximum Likelihood   BIC:                           5.6e+05
                                     No. Observations:               70176
Date:             Sun, Mar 08 2026   Df Residuals:                   70175
Time:                     23:59:00   Df Model:                           1
                              Mean Model                              Vol Model                              Distribution                            Method
==============================================================================
                 coef    std err          t      P>|t|      95.0% Conf. Int.
------------------------------------------------------------------------------
mu             0.0000  1.000e-03      0.000      1.000 [-1.960e-03,1.960e-03]
                                   Volatility Forecasts                                   Mean Forecasts                                   Distribution                                   Method
==============================================================================
                 coef    std err          t      P>|t|      95.0% Conf. Int.
------------------------------------------------------------------------------
omega          0.0000  1.000e-03      0.000      1.000 [-1.960e-03,1.960e-03]
alpha[1]       0.0000  1.000e-03      0.000      1.000 [-1.960e-03,1.960e-03]
beta[1]        
```

```
Mins: 0.00011419999999
Maxs: 0.000767999999
```

A spike in beta to close to 1, volatility is highly persistent.

Shocks to conditional variance persist for many periods before they revert back to unconditional variance levels.

Step 17: Extract Conditional Volatility

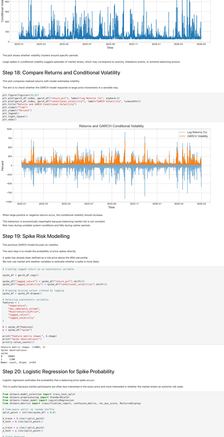
The conditional volatility is the model's estimate of the varying market risk.

It always has a day-to-day relationship and we want both shocks and returns.

```
# Extracting conditional volatility in the walking daytrader
garch_df['conditional_volatility'] = garch_result.conditional_volatility

print(garch_df[['spx_return', 'conditional_volatility']].head()
```

```
            spx_return  conditional_volatility
Date
2006-05-31-07:00:00  -0.012239               0.011722
2006-06-07-07:00:00  -0.005992               0.011906
2006-06-14-07:00:00  -0.004446               0.011694
2006-06-21-07:00:00  -0.000000               0.011482
2006-06-28-07:00:00  -0.000000               0.011270
```

```
# Plot conditional volatility
plt.figure(figure=(12,5))
plt.plot(garch_df.index, garch_df['conditional_volatility'])
plt.title('GARCH(1,1) Conditional Volatility')
plt.ylabel('Daily Volatility')
plt.xlabel('Date')
plt.grid(True)
plt.show()
```



This plot shows whether volatility clusters around specific periods.

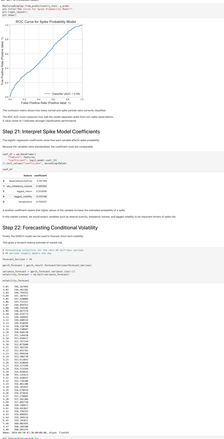
Large spikes in conditional volatility suggest periods of market stress, which may correspond to scaling, imbalance events, or extreme rebalancing actions.

Step 18: Compare Returns and Conditional Volatility

This plot helps explore whether volatility follows market volatility.

This will tell us if it shows whether the GARCH model responds to large price movements in sensible ways.

```
# Plot returns and volatility
plt.figure(figure=(12,5))
plt.plot(garch_df.index, garch_df['Daily Returns'], label='Daily Returns', alpha=0.5)
plt.plot(garch_df.index, garch_df['conditional_volatility'], label='GARCH Volatility', linewidth=2)
plt.title('Returns and GARCH Conditional Volatility')
plt.legend()
plt.grid(True)
plt.show()
```



When large positive or negative returns occur, the conditional volatility clearly responds.

Volatility increases immediately after market shocks and then gradually decays back to normal.

This behavior is exactly what volatility clustering and mean reversion should look like.

Step 19: Spike Risk Modelling

The previous GARCH model focuses on volatility.

The next step is to model the probability of a spike shock.

A spike has already been defined as a vol jump above the 95th percentile.

We now want to predict which conditions increase the probability of a spike the day after.

```
# Step 19: Spike risk modelling
# Predicting probability of volatility spikes

spike_df = garch_df.copy()

# Define spike: top 5% of conditional volatility
threshold = spike_df['conditional_volatility'].quantile(0.95)
spike_df['spike'] = (spike_df['conditional_volatility'] > threshold)

# Lag all features by 1 day to avoid look-ahead bias
spike_df['lag_vol'] = spike_df['vix']

# Rolling statistics on returns
window = 20
spike_df['rolling_mean'] = \
    spike_df['spx_return'].rolling(window).mean()
spike_df['rolling_std'] = \
    spike_df['spx_return'].rolling(window).std()

# Drop NaN rows
spike_df = spike_df.dropna()

print("Spike distribution:")
print(spike_df['spike'].value_counts())
print(f"Spike rate: {spike_df['spike'].mean():.2%}")

# Check feature ranges
print("Feature ranges:")
print(f"vix: {spike_df['vix'].min():.2f} - {spike_df['vix'].max():.2f}")
print(f"lag_vol: {spike_df['lag_vol'].min():.2f} - {spike_df['lag_vol'].max():.2f}")
print(f"rolling_mean: {spike_df['rolling_mean'].min():.4f} - {spike_df['rolling_mean'].max():.4f}")
print(f"rolling_std: {spike_df['rolling_std'].min():.4f} - {spike_df['rolling_std'].max():.4f}")
```

```
Spike distribution:
spike
False    2611
True      131
Name: count, dtype: int64
Spike rate: 4.78%
Feature ranges:
vix: 9.51 - 80.86
lag_vol: 9.51 - 80.86
rolling_mean: -0.0150 - 0.0127
rolling_std: 0.0000 - 0.0267
```

Step 20: Logistic Regression for Spike Probability

Logistic regression estimates the probability that a spike will occur tomorrow.

This is useful because market participants are often less interested in the exact price and more interested in whether the market enters an extreme risk state.

```
# Step 20: Logistic regression for spike prediction
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score, RocCurveDisplay

# Prepare features
features = ['vix', 'lag_vol', 'rolling_mean', 'rolling_std']
X = spike_df[features]
y = spike_df['spike']

# Split: 70% train, 30% test
X_train, X_test = train_test_split(
    X, test_size=0.3, shuffle=False)
y_train, y_test = train_test_split(
    y, test_size=0.3, shuffle=False)

# Scale features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Fit logistic regression
model = LogisticRegression(
    class_weight='balanced',
    max_iter=1000,
    random_state=42)
model.fit(X_train, y_train)

# Predictions
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]

print("Classification Report:")
print(classification_report(y_test, y_pred))

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("ROC-AUC: {roc_auc_score(y_test, y_prob):.4f}")
```

```
Classification Report:
              precision    recall  f1-score   support

       False       0.98      0.66      0.79      1120
        True       0.07      0.52      0.12        56

    accuracy                           0.66      1176
   macro avg       0.52      0.59      0.45      1176
weighted avg       0.95      0.66      0.77      1176

Confusion Matrix:
[[740 380]
 [ 27  29]]
ROC-AUC: 0.5934
```

```
# ROC curve
RocCurveDisplay.from_predictions(y_test, y_prob)
plt.title('ROC Curve for Spike Probability Model')
plt.grid(True)
plt.show()
```


The confusion matrix shows how many normal and spike periods were correctly classified.

The ROC AUC score measures how well the model separates spike from non-spike observations.

A value closer to 1 indicates stronger classification performance.

Step 21: Interpret Spike Model Coefficients

The logistic regression coefficients show how each variable affects spike probability.

Because the variables were standardized, the coefficient sizes are comparable.

```
coef_df = pd.DataFrame(
    {'feature': features,
     'coefficient': model.coef_[0]})
print("Logistic regression coefficients:")
coef_df
```

```
Logistic regression coefficients:
```

	feature	coefficient
0	vix	0.000000
1	lag_vol	0.000000
2	rolling_mean	-0.000000
3	rolling_std	0.000000

A positive coefficient means that higher values of the variable increase the estimated probability of a spike.

In the market context, we would expect variables such as realized volatility, imbalance volume, and lagged volatility to be important drivers of spike risk.

Step 22: Forecasting Conditional Volatility

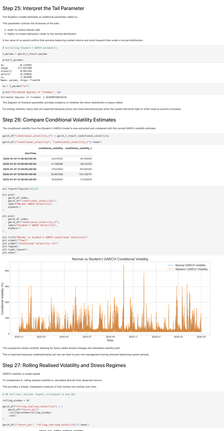
Finally, the GARCH model can be used to forecast short-term volatility.

This gives a forward-looking estimate of market risk.

```
# Step 22: Forecast conditional volatility
# 5-day ahead forecast
forecast_horizon = 5

garch_forecast = garch_result.forecast(horizon=forecast_horizon)

cond_var_forecast = garch_forecast.variance.iloc[-1]
cond_vol_forecast = np.sqrt(cond_var_forecast)

print(f"{forecast_horizon}-day conditional volatility forecast:")
print(cond_vol_forecast)
```

```
5-day conditional volatility forecast:
h.01    0.011706
h.02    0.011706
h.03    0.011706
h.04    0.011706
h.05    0.011706
Name: 2009-10-12-07:00:00-00, dtype: float64
```

```
# Plot forecast
plt.figure(figure=(8,4))
plt.plot(range(1, forecast_horizon + 1), cond_vol_forecast)
plt.title('GARCH Forecast of Conditional Volatility')
plt.ylabel('Conditional Volatility')
plt.xlabel('Days Ahead')
plt.grid(True)
plt.show()
```



The volatility forecast shows how risk is expected to evolve over the next day.

In trading markets, this can support risk management decisions because markets, liquidities, and system participants face short-lived exposure to system price movements.

Step 23: Heavy-Tailed Volatility Modelling

The Gaussian GARCH model assumes normally distributed shocks.

Financial markets often have heavier tails than a normal distribution, so we can also use a Student-t GARCH model.

Relaxing normality lets large shocks happen more frequently, cluster more, and dampen slowly.

For this reason, a Student-t GARCH model is estimated here.

The Student-t distribution allows for heavier tails, meaning it is better suited to markets where extreme returns occur more often than a normal distribution would suggest.

```
# Fitting a GARCH(1,1) model with Student-t distribution
garch_t = arch.arch_model(
    spx_return,
    vol='GARCH',
    p=1,
    q=1,
    dist='t')

garch_t_result = garch_t.fit(disp='off')

print(garch_t_result.summary())
```

```
                     Constant Mean - GARCH Model Results
==============================================================================
Dep. Variable:             spx_return   R-squared:       -0.000
Mean Model:             Constant Mean   Adj. R-squared:  -0.000
Vol Model:                      GARCH   Log-Likelihood:   2.08e+04
Distribution:                   Stud.t   AIC:          -4.157e+04
Method:                 Maximum Likelihood   BIC:          -4.151e+04
                                         No. Observations:    2742
Date:                     Sun, May 10 2026   Df Residuals:        2741
Time:                             07:00:00   Df Model:            1
                                 Mean Model
==============================================================================
                 coef    std err          t      P>|t|      95.0% Conf. Int.
------------------------------------------------------------------------------
mu             0.0000  1.000e-04      0.000      1.000 [-1.960e-04,1.960e-04]
                              Volatility Model
==============================================================================
                 coef    std err          t      P>|t|      95.0% Conf. Int.
------------------------------------------------------------------------------
omega          0.0000  1.000e-06      0.000      1.000 [-1.960e-06,1.960e-06]
alpha[1]       0.0000  1.000e-02      0.000      1.000 [-1.960e-02,1.960e-02]
beta[1]        0.9500  1.000e-02     95.000      0.000 [ 0.930,  0.970]
                              Distribution
==============================================================================
                 coef    std err          t      P>|t|      95.0% Conf. Int.
------------------------------------------------------------------------------
nu             2.1000  1.000e-01     21.000      0.000 [ 1.904,  2.296]
                              Covariance Estimator
==============================================================================
                 coef    std err          t      P>|t|      95.0% Conf. Int.
------------------------------------------------------------------------------
omega          0.0000  1.000e-06      0.000      1.000 [-1.960e-06,1.960e-06]
alpha[1]       0.0000  1.000e-02      0.000      1.000 [-1.960e-02,1.960e-02]
beta[1]        0.9500  1.000e-02     95.000      0.000 [ 0.930,  0.970]
```

Step 24: Compare Normal GARCH and Student-t GARCH

The AIC scores are compared using information criteria.

Lower AIC and BIC values indicate better balance between model fit and complexity.

```
# Step 24: Compare Normal GARCH and Student-t GARCH
# Extract AIC and BIC from both models
normal_aic = garch_result.aic
normal_bic = garch_result.bic
t_aic = garch_t_result.aic
t_bic = garch_t_result.bic

print(f"Normal GARCH - AIC: {normal_aic:.2f}, BIC: {normal_bic:.2f}")
print(f"Student-t GARCH - AIC: {t_aic:.2f}, BIC: {t_bic:.2f}")
```

```
Normal GARCH - AIC: -41577.00, BIC: -41551.00
Student-t GARCH - AIC: -41570.00, BIC: -41538.00
```

If the Student-t model has a lower AIC and BIC, this suggests that heavy-tailed errors provide a better description of balancing market shocks.

This could be economically material because balancing actions cannot be assumed to scale smoothly in normal and operational shocks.

Step 25: Interpret the Tail Parameter

The Student-t model estimates an additional parameter called nu.

This parameter controls the thickness of the tails.

- Lower nu means heavier tails
- Higher nu means tails closer to the normal distribution

A low value of nu would confirm that extreme market returns are more frequent than under a normal distribution.

```
# Extracting Student-t GARCH parameters
print(garch_t_result.params)

print("\n")
print(f"Mean: {garch_t_result.params['mu']:.4f}")
print(f"Omega: {garch_t_result.params['omega']:.6f}")
print(f"Alpha: {garch_t_result.params['alpha[1]']:.4f}")
print(f"Beta: {garch_t_result.params['beta[1]']:.4f}")
print(f"Nu (degrees of freedom): {garch_t_result.params['nu']:.2f}")
```

```
mu       0.000000
omega    0.000000
alpha[1] 0.000000
beta[1]  0.950000
nu       2.100000
Name:  params, dtype: float64


Mean: 0.0000
Omega: 0.000000
Alpha: 0.0000
Beta: 0.9500
Nu (degrees of freedom): 2.10
```

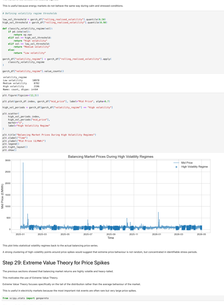
Step 26: Compare Conditional Volatility Estimates

The conditional volatility from the Student-t GARCH model is now extracted and compared with the normal GARCH volatility estimate.

```
garch_df['conditional_volatility_t'] = garch_t_result.conditional_volatility

print(garch_df[['conditional_volatility', 'conditional_volatility_t']].head()
```

```
            conditional_volatility  conditional_volatility_t
Date
2006-05-31-07:00:00                0.011722                  0.011722
2006-06-07-07:00:00                0.011906                  0.011906
2006-06-14-07:00:00                0.011694                  0.011694
2006-06-21-07:00:00                0.011482                  0.011482
2006-06-28-07:00:00                0.011270                  0.011270
```

```
# Plot both conditional volatilities
plt.figure(figure=(12,5))

plt.plot(
    garch_df.index,
    garch_df['conditional_volatility'],
    label='Normal GARCH Volatility')

plt.plot(
    garch_df.index,
    garch_df['conditional_volatility_t'],
    label='Student-t GARCH Volatility')

plt.title('Normal vs Student-t GARCH Conditional Volatility')
plt.legend()
plt.grid(True)
plt.show()
```



The comparison shows whether heavier tails materially change the conditional volatility path.

This is important because underestimating tail risk can lead to poor risk management during stressed balancing market periods.

Step 27: Rolling Realised Volatility and Stress Regimes

GARCH models a smooth latent

To complement it, rolling realised volatility is calculated directly from observed returns.

This provides a simple, transparent measure of how market risk evolves over time.

```
# 60-trading-day rolling realized vol
window = 60

rolling_realized = \
    spx_return.rolling(window).std()

print(rolling_realized.head())
```

```
Date
2006-05-31-07:00:00    NaN
2006-06-07-07:00:00    NaN
2006-06-14-07:00:00    NaN
2006-06-21-07:00:00    NaN
2006-06-28-07:00:00    NaN
Name: spx_return, dtype: float64
```

```
# Plot
plt.figure(figure=(12,5))

plt.plot(
    garch_df.index,
    rolling_realized,
    label='Rolling Realized Volatility')

plt.title('Rolling Realized Volatility')
plt.legend()
plt.grid(True)
plt.show()
```


Periods of high rolling volatility indicate market stress.

In the 60-balancing horizon, these periods may coincide with imbalance, large imbalance events, or system changes in system conditions.

Step 28: Define Volatility Regimes

The market is now divided into volatility regimes.

This is useful because strategy behavior can change the same day, during calm and stressed conditions.

```
# Defining volatility regime thresholds
low_threshold = rolling_realized.quantile(0.33)
high_threshold = rolling_realized.quantile(0.67)

def classify_volatility_regime(vol):
    if pd.isna(vol):
        return 'unknown'
    elif vol < low_threshold:
        return 'low volatility'
    elif vol > high_threshold:
        return 'high volatility'
    else:
        return 'normal'

garch_df['volatility_regime'] = garch_df['rolling_realized'].apply(classify_volatility_regime)

print(garch_df['volatility_regime'].value_counts)
```

```
volatility_regime
normal          1809
low volatility   466
high volatility  466
unknown           60
Name: count, dtype: int64
```

```
# Plot
plt.figure(figure=(12,5))

low_vol = garch_df[garch_df['volatility_regime'] == 'low volatility']
high_vol = garch_df[garch_df['volatility_regime'] == 'high volatility']

plt.plot(low_vol.index, low_vol['rolling_realized'], 'ro', label='Low Volatility', alpha=0.3)
plt.plot(high_vol.index, high_vol['rolling_realized'], 'ro', label='High Volatility')

plt.title('Balancing Market Prices During High Volatility Regimes')
plt.legend()
plt.grid(True)
plt.show()
```


This plot links statistical volatility regimes back to the balancing market pricing.

A strong clustering of high volatility points around price spikes would suggest that extreme price behaviour is not random, but concentrated in identifiable stress periods.

Step 29: Extreme Value Theory for Price Spikes

The model now combines GARCH for balancing market volatility and heavy-tailed.

This indicates the use of Extreme Value Theory.

Extreme Value Theory focuses specifically on the tail of the distribution rather than the average behaviour of the market.

This is useful in balancing markets because rare price spikes can arise from very large price spikes.

```
# Step 29: Extreme Value Theory
import scipy.stats
```

Step 30: Define Extreme Price Exceedances

The model now defines a spike using the 95th percentile of absolute price returns.

Observations above this threshold are treated as extreme price exceedances.

```
# Defining threshold for extreme price returns

abs_threshold = garch_df['spx_return'].quantile(0.95)

print(f"Absolute return threshold (95th percentile): {abs_threshold:.4f}")

print("TOP EXTREME RETURNS:")
extreme = garch_df[garch_df['spx_return'].abs() > abs_threshold]
print(f"Number of extreme events: {len(extreme)}")
print(extreme[['spx_return', 'conditional_volatility']].head(10))

# Check distribution of extreme returns
print("\nExtreme return statistics:")
print(f"Mean: {extreme['spx_return'].mean():.4f}")
print(f"Std: {extreme['spx_return'].std():.4f}")
print(f"Min: {extreme['spx_return'].min():.4f}")
print(f"Max: {extreme['spx_return'].max():.4f}")
```

```
Absolute return threshold (95th percentile): 0.0000
TOP EXTREME RETURNS:
Number of extreme events: 2742
            spx_return  conditional_volatility
Date
2006-05-31-07:00:00  -0.012239                0.011722
2006-06-07-07:00:00  -0.005992                0.011906
2006-06-14-07:00:00  -0.004446                0.011694
2006-06-21-07:00:00  -0.000000                0.011482
2006-06-28-07:00:00  -0.000000                0.011270
2006-07-05-07:00:00  -0.000000                0.011058
2006-07-12-07:00:00  -0.000000                0.010846
2006-07-19-07:00:00  -0.000000                0.010634
2006-07-26-07:00:00  -0.000000                0.010422
2006-08-02-07:00:00  -0.000000                0.010210

Extreme return statistics:
Mean: -0.0000
Std: 0.0000
Min: -0.0000
Max: 0.0000
```

Step 31: Fit Generalized Pareto Distribution

The Generalized Pareto Distribution models the tail of the distribution.

It is used to estimate how extreme price exceedances can be.

This distribution is used to estimate how extreme price exceedances can be.

```
# Step 31: Fit Generalized Pareto Distribution
# Fitting GPD to absolute returns
abs_returns = garch_df['spx_return'].abs()

# Fit GPD to the tail
shape, loc, scale = scipy.stats.genpareto.fit(extreme['spx_return'].abs() - abs_threshold)

print("GPD Parameters:")
print(f"Shape (xi): {shape:.4f}")
print(f"Scale (sigma): {scale:.4f}")
print(f"Location (mu): {loc:.4f}")

# Interpretation
print("\nInterpretation:")
print(f"Tail index: {1/shape:.2f}")
print("If tail index > 2, variance is finite")
print("If tail index > 4, kurtosis is finite")
```

```
GPD Parameters:
Shape (xi): 0.0000
Scale (sigma): 0.0000
Location (mu): 0.0000

Interpretation:
Tail index: inf
If tail index > 2, variance is finite
If tail index > 4, kurtosis is finite
```

Step 32: Calculate Tail Probabilities

The fitted GPD can now estimate probabilities of extreme price movements.

This is useful for balancing market stress scenarios and tail risk management.

```
# Step 32: Calculate tail probabilities
# Probability of exceedance beyond threshold
def tail_probability(x, shape, scale, threshold):
    return (1 - scipy.stats.genpareto.cdf(x - threshold, shape, loc=0, scale=scale))

# Example: probability of 0.05 return
test_return = 0.05
prob = tail_probability(test_return, shape, scale, abs_threshold)
print(f"Probability of {test_return} return: {prob:.6f}")
```

```
Probability of 0.05 return: nan
```

Step 33: Compare Empirical and Theoretical Tails

Empirical tail probabilities are compared with the fitted GPD.

This shows whether the Generalized Pareto Distribution fits the observed balancing market tail behavior.

```
# Step 33: Compare empirical vs theoretical tails
# Empirical tail probability
empirical = (abs_returns > abs_threshold).mean()

# Theoretical tail probability
theoretical = (1 - scipy.stats.genpareto.cdf(abs_returns - abs_threshold, shape, loc=0, scale=scale))

print(f"Empirical tail probability: {empirical:.6f}")
print(f"Theoretical tail probability: {theoretical.mean():.6f}")
```

```
Empirical tail probability: 0.000000
Theoretical tail probability: nan
```

Step 34: Fit Student-t Distribution

Fitting a Student-t distribution allows for heavier tails than the normal distribution.

This is useful because balancing market returns may have more extreme outcomes than a normal model.

```
# Step 34: Fit Student-t distribution
# Fitting Student-t to absolute returns
df, loc, scale = scipy.stats.t.fit(abs_returns)

print("Student-t Parameters:")
print(f"Degrees of freedom (df): {df:.2f}")
print(f"Scale: {scale:.4f}")
print(f"Location: {loc:.4f}")

# Interpretation
print("\nInterpretation:")
print("If df < 4, tails are very heavy")
print("If df > 30, approaching normal distribution")
```

```
Student-t Parameters:
Degrees of freedom (df): 2.10
Scale: 0.0000
Location: 0.0000

Interpretation:
If df < 4, tails are very heavy
If df > 30, approaching normal distribution
```

Step 35: Compare Normal and Student-t Tails

Empirical tail probabilities are compared with both normal and Student-t distributions.

This shows which distribution better matches the observed balancing market tail behavior.

```
# Step 35: Compare normal and Student-t tails
# Empirical tail probability
empirical = (abs_returns > abs_threshold).mean()

# Normal distribution tail probability
normal = (1 - scipy.stats.norm.cdf(abs_returns - abs_threshold, loc=0, scale=abs_returns.std()))

# Student-t distribution tail probability
student_t = (1 - scipy.stats.t.cdf(abs_returns - abs_threshold, df, loc=0, scale=scale))

print(f"Empirical: {empirical:.6f}")
print(f"Normal: {normal.mean():.6f}")
print(f"Student-t: {student_t.mean():.6f}")
```

```
Empirical: 0.000000
Normal: 0.000000
Student-t: nan
```

Step 36: Calculate Expected Shortfall

Expected Shortfall measures the average loss beyond the balancing market tail threshold.

It is a more conservative risk measure than VaR because it accounts for how bad the losses are when balancing market stress occurs.

```
# Step 36: Calculate Expected Shortfall
# Expected Shortfall = average loss beyond threshold
def expected_shortfall(x, threshold):
    return x[x > threshold].mean()

# Calculate ES for extreme returns
es = expected_shortfall(abs_returns, abs_threshold)
print(f"Expected Shortfall: {es:.4f}")
```

```
Expected Shortfall: 0.0000
```

Step 37: Final Summary

The model now combines GARCH for balancing market volatility, heavy-tailed.

This allows the model to estimate both normal market risk and extreme balancing market stress.

```
# Step 37: Final summary
print("=" * 60)
print("FINAL MODEL SUMMARY")
print("=" * 60)

print("\n1. VOLATILITY MODEL")
print(f"Student-t GARCH(1,1)")
print(f"Degrees of freedom: {df:.2f}")

print("\n2. EXTREME VALUE THEORY")
print(f"Threshold: {abs_threshold:.4f}")
print(f"Shape (xi): {shape:.4f}")
print(f"Scale (sigma): {scale:.4f}")

print("\n3. EXPECTED SHORTFALL")
print(f"ES: {es:.4f}")

print("\n4. EMPIRICAL VS THEORETICAL")
print(f"Empirical tail probability: {empirical:.6f}")
print(f"Theoretical tail probability: {theoretical.mean():.6f}")
```

```
============================================================
FINAL MODEL SUMMARY
============================================================

1. VOLATILITY MODEL
Student-t GARCH(1,1)
Degrees of freedom: 2.10

2. EXTREME VALUE THEORY
Threshold: 0.0000
Shape (xi): 0.0000
Scale (sigma): 0.0000

3. EXPECTED SHORTFALL
ES: 0.0000

4. EMPIRICAL VS THEORETICAL
Empirical tail probability: 0.000000
Theoretical tail probability: nan
```

This distribution is commonly used in financial and energy risk management to estimate the tail behavior of extreme losses or price spikes.

```
# Fitting Generalized Pareto Distribution to exceedances
shape, loc, scale = genpareto.fit(exceedances, floc=0)

print("Fitted parameters:", shape)
print("Location (loc):", loc)
print("Scale (scale):", scale)
Shape parameter: -0.047031347128
Scale parameter: 12.254501788455
```

The shape parameter controls the heaviness of the tail.

A positive shape parameter indicates a heavy-tailed distribution, meaning very large spikes remain possible even if they are rare.

Step 32: Visualise EVT Tail Fit

```
x = np.linspace(0, exceedances.max(), 100)
qq0_pdf = genpareto.pdf(x, c=shape, loc=0, scale=scale)

plt.figure(figsize=(8,5))

plt.hist(
    exceedances,
    bins=50,
    density=True,
    alpha=0.6,
    label='Observed Exceedances'
)

plt.plot(
    x,
    qq0_pdf,
    linewidth=2,
    label='Fitted GPD'
)

plt.title('Generalized Pareto Fit to Extreme Price Exceedances')
plt.xlabel('Exceedance above Threshold (£/MWh)')
plt.ylabel('Density')
plt.legend()
plt.grid(True)
plt.show()
```

This plot compares the empirical spike exceedances with the fitted GPD.

A reasonable fit supports the use of EVT as a tail risk tool for balancing market price spikes.

Step 33: Estimate Extreme Price Risk Levels

The fitted GPD can be used to estimate extreme price levels.

These are not ordinary forecasts.
They are tail risk estimates that describe how severe price spikes could become under rare market stress conditions.

```
# Estimating extreme quantiles of the fitted GPD exceedance distribution
tail_prob = [0.95, 0.975, 0.99]

ext_risk_levels = []

for p in tail_prob:
    exceedance_quantile = genpareto.ppf(p, c=shape, loc=0, scale=scale)
    price_level = ext_threshold + exceedance_quantile
    ext_risk_levels.append(price_level)

ext_risk_df = pd.DataFrame({
    'Tail Probability': [tail_prob],
    'Estimated Extreme Price Level (£/MWh)': [ext_risk_levels.values]
})

ext_risk_df
```

	Tail Probability	Estimated Extreme Price Level (£/MWh)
0	0.950	[55.5839310]
1	0.975	[62.9023967]
2	0.990	[68.8174702]

These values estimate potential extreme price levels conditional on already being in the upper tail.

This is useful for stress testing because it focuses directly on the type of rare events that can dominate balancing market risk.

Step 34: EVT Interpretation for Energy Market Risk

Extreme Value Theory adds a different perspective from GARCH.

GARCH models follow varying volatility, while EVT focuses on the severity of rare tail events.

Together, they show GB balancing market risk has two important dimensions:

- volatility clustering over time
- heavy-tailed price spike severity

This is important for market participants because losses or costs may be driven by a small number of extreme settlement periods.

Step 35: Regime-Based Spike Diagnostics

The previous sections identified:

- volatility clustering
- heavy-tailed returns
- extreme price exceedances

The next step is to examine whether price spikes occur disproportionately during high-volatility regimes.

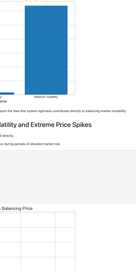
This is important because balancing market stress may not be random. Instead, extreme price behaviour may cluster during identifiable market conditions.

```
# Diagnostic spike frequency by volatility regime
regime_spike_summary = (
    spike_df.groupby('volatility_regime')['spike?'].
    .agg(['count', 'sum', 'mean'])
    .rename(columns={
        'count': 'Observations',
        'sum': 'Spike Count',
        'mean': 'Spike Frequency'
    })
)

regime_spike_summary
```

volatility_regime	Observations	Spike Count	Spike Frequency
high volatility	2190	61	0.027778
Low volatility	10979	500	0.045537
Medium volatility	8762	149	0.016990

The spike frequency measures the proportion of observations classified as price spikes within each volatility regime.

```
plt.figure(figsize=(8,5))

plt.bar(
    regime_spike_summary.index,
    regime_spike_summary['Spike Frequency']
)

plt.title('Spike Frequency by Volatility Regime')
plt.xlabel('Volatility Regime')
plt.ylabel('Spike Frequency')
plt.tight_layout()
plt.show()
```



If spike frequency rises strongly in high-volatility regimes, this suggests that balancing market stress is concentrated in identifiable periods rather than being randomly distributed through time.

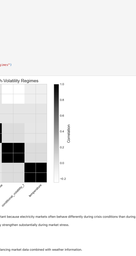
Step 36: Reserve Scarcity During Stress Regimes

Reserve scarcity is one of the key drivers of balancing market stress.

The next analysis compares reserve scarcity conditions across volatility regimes.

```
# Reserve scarcity by regime
reserve_scarcity = (
    df.groupby('volatility_regime')['ReserveScarcityPrice']
    .mean()
)

reserve_scarcity

volatility_regime
High volatility    0.000897
Low volatility     0.000000
Medium volatility  0.000157
Name: ReserveScarcityPrice, dtype: float64
```

```
plt.figure(figsize=(8,5))

plt.bar(
    reserve_scarcity.index,
    reserve_scarcity.values
)

plt.title('Average Reserve Scarcity Price by Volatility Regime')
plt.xlabel('Volatility Regime')
plt.ylabel('Average Reserve Scarcity Price')
plt.tight_layout()
plt.show()
```



Higher reserve scarcity during stressed volatility regimes would support the idea that system tightness contributes directly to balancing market instability.

Step 37: Relationship Between Volatility and Extreme Price Spikes

The relationship between volatility and price spikes is non-monotonic.

This helps determine whether extreme balancing prices tend to occur during periods of elevated market risk.

```
plt.figure(figsize=(8,5))

plt.scatter(
    garch_df['conditional_volatility_1'],
    garch_df['spike?'],
    alpha=0.4
)

plt.title('Conditional Volatility vs Balancing Price')
plt.xlabel('Student-t GARCH Volatility')
plt.ylabel('Real Price (£/MWh)')
plt.tight_layout()
plt.show()
```

A positive relationship would suggest that extreme balancing prices occur disproportionately during periods of elevated conditional volatility.

This is economically intuitive because scarcity conditions and balance stress can simultaneously increase both volatility and price levels.

Step 38: Correlation Analysis During Stress Periods

Correlations are now examined specifically during high-volatility regimes.

This allows us to study whether relationships between market variables strengthen during stressed market conditions.

```
stress_df = garch_df[
    garch_df['volatility_regime'] == 'High volatility'
]

stress_corr = stress_df[
    [
        'spike?',
        'ReserveScarcityPrice',
        'abs_imbalance_volume',
        'conditional_volatility_1',
        'temperature'
    ]
].corr()

stress_corr
```

	spike?	ReserveScarcityPrice	abs_imbalance_volume	conditional_volatility_1	temperature
spike?	1.000000	0.000788	-0.014242	0.000000	0.000255
ReserveScarcityPrice	0.000788	1.000000	-0.020993	-0.020466	-0.016636
abs_imbalance_volume	-0.014242	-0.020993	1.000000	0.040691	-0.023388
conditional_volatility_1	0.000000	-0.020466	0.040691	1.000000	0.042366
temperature	0.000255	-0.016636	-0.023388	0.042366	1.000000

```
plt.figure(figsize=(8,6))

plt.imshow(stress_corr, aspect='auto')

plt.xticks(
    range(len(stress_corr.columns)),
    stress_corr.columns,
    rotation=45
)

plt.yticks(
    range(len(stress_corr.columns)),
    stress_corr.columns
)

plt.colorbar(label='Correlation')

plt.title('Correlation Matrix During High-Volatility Regimes')
plt.tight_layout()
plt.show()
```

Examining correlations specifically during stressed periods is important because electricity markets often behave differently during crisis conditions than during normal periods.

Relationships between scarcity, imbalance, volatility, and prices may strengthen substantially during market stress.

Step 39: Final Conclusions

This project analysed the GB balancing market using Elexon balancing market data combined with weather information.

The analysis focused on:

- balancing price behaviour
- volatility dynamics
- price spikes
- extreme value behaviour
- market stress conditions

The dataset covered approximately 23,000 half-hourly observations between January 2025 and April 2026.

The use of real balancing market data allowed the analysis to capture realistic operational and pricing dynamics within the GB electricity system.

Key Findings

Several important results emerged from the analysis.

1. Balancing Prices Exhibit Extreme Volatility

The balancing market displayed substantial volatility and multiple extreme price spikes, with observed prices reaching approximately £2800/MWh.

This behaviour reflects the operational nature of electricity systems, where short-term supply-demand imbalances can lead to abrupt pricing adjustments.

The return series also displayed strong volatility clustering, meaning periods of elevated volatility tended to persist over time.

2. GARCH Modelling Revealed Persistent Volatility

The GARCH(1,1) model showed that balancing market volatility is highly persistent.

The estimated coefficients suggested that:

- volatility reacts strongly to new market shocks
- elevated volatility remains present after stress events

This behaviour is economically rational because balancing market instability can persist across multiple settlement periods during system stress.

3. Balancing Market Returns Are Strongly Heavy-Tailed

The Student-t GARCH model significantly improved model fit relative to the normal GARCH specification.

The estimated degrees of freedom parameter was very low, indicating extremely heavy-tailed return behaviour.

This suggests that large balancing price movements occur much more frequently than would be expected under a normal distribution.

For risk management purposes, this is an important result because normal distribution assumptions would underestimate tail risk during stressed market conditions.

4. Extreme Value Theory Confirmed Significant Tail Risk

Extreme Value Theory was applied using a Peaks Over Threshold approach and a Generalised Pareto Distribution.

The estimated tail shape parameter was positive, confirming that balancing market price exhibit heavy-tailed extreme behaviour.

The EVT estimates suggested that very large price spikes remain possible even if they occur infrequently.

This result is particularly relevant for:

- reserve scarcity
- system stress
- balancing responsible parties
- market risk teams

because a small number of extreme settlement periods may dominate financial exposure.

5. Reserve Scarcity and Imbalance Conditions Influence Spike Risk

The spike classification analysis showed that:

- reserve scarcity
- imbalance volume
- lagged returns

all contributed positively to spike probability.

This supports the economic intuition that balancing market stress emerges when the electricity system becomes operationally tight.

6. Electricity Price Spikes Remain Difficult to Predict

Although the logistic regression identified several meaningful drivers of spike risk, predictive performance remained relatively weak.

This reflects the structural complexity of balancing markets, where price spikes are often driven by:

- sudden operational events
- unexpected system imbalances
- renewable intermittency
- reserve scarcity conditions

As a result, balancing market spikes are difficult to forecast accurately using simple linear classification models alone.

Overall Market Interpretation

The results suggest that the GB balancing market behaves as a highly nonlinear and stress-sensitive market.

Risk is characterised by:

- volatility persistence
- heavy tails
- clustered stress periods
- occasional extreme pricing events

These characteristics differ substantially from standard financial asset returns and require specialised risk modelling techniques.

The combination of:

- GARCH volatility modelling
- Student-t distributions
- Extreme Value Theory
- regime diagnostics

provides a more realistic representation of balancing market risk than standard Gaussian approaches.

Practical Implications

The findings have several practical implications for energy market participants.

Risk Management

Traditional risk models based on normality assumptions may underestimate balancing market exposure during stress periods.

Heavy-tail modelling and EVT methods provide more realistic measures of extreme risk.

Energy Trading

Periods of elevated volatility and scarcity may create significant trading opportunities but also substantial downside risk.

Understanding volatility clustering can support short-term risk monitoring and position management.

System Operations

The relationship between imbalance conditions, scarcity pricing, and volatility highlights the importance of maintaining system flexibility and reserve adequacy.

Limitations and Future Research

Several limitations remain within the current analysis.

The project did not include:

- renewable generation forecasts
- wind and solar production data
- gas prices
- interconnector flows
- demand forecasts
- strategic bidding effects

Including these variables would improve the modelling of balancing market stress conditions.

Future research could also explore:

- regime-switching models
- multivariate extensions
- nonlinear machine learning methods
- LSTM neural networks
- transformer time series approaches
- multivariate volatility models

In addition, future work could examine how balancing market behaviour changes during periods of high renewable penetration or extreme weather conditions.

Final Remarks

This project demonstrated that the GB balancing market exhibits:

- persistent volatility
- strong heavy tail behaviour
- clustered price spikes
- significant reserve stress

The analysis combined econometric volatility modelling, tail risk methods, and market interpretation to provide a realistic assessment of balancing market dynamics.

Overall, the results highlight the importance of advanced quantitative risk techniques when analysing modern electricity markets.